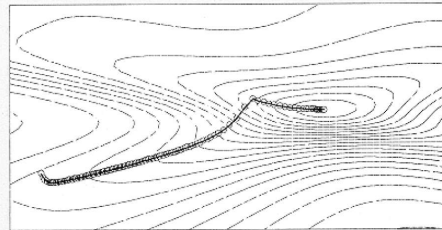


Algoritmos de Aprendizado

- Regra de Hebb
- Perceptron
- Delta Rule (Least Mean Square)
- **Multi-Layer Perceptrons (Back Propagation)**
- Hopfield
- Competitive Learning
- Radial Basis Functions (RBFs)

Aprendizado Batch x Incremental

• Aprendizado em Batch (por ciclo)



Melhor aproximação do verdadeiro gradiente decrescente

Formas de Aprendizado

- Existe dois métodos básicos de aplicação do algoritmo Back Propagation:
 - ✓ **Aprendizado em Batch** ("Batch Learning", por ciclo, etc)
 - ✓ **Aprendizado Incremental** ("on-line", "pattern-mode", por padrão, etc)

Aprendizado Incremental

• Aprendizado Incremental (por padrão)

- **Atualiza os pesos a cada apresentação de um novo padrão**
 - ⇒ os pesos são atualizados usando o gradiente do erro de **um único padrão**
 - ⇒ não é mais uma aproximação simples do gradiente decrescente
 - ⇒ mais atualizações ocorrem em um mesmo período de tempo
 - ⇒ tende a aprender melhor o último padrão apresentado ⇒ **sequência aleatória**

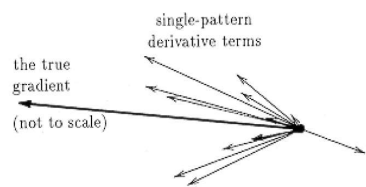
Aprendizado Batch x Incremental

• Aprendizado em Batch (por ciclo)

- **Somente ajusta os pesos após a apresentação de TODOS os padrões**
- **Cada padrão é avaliado com a MESMA configuração de pesos**
- ⇒ obtém-se os termos derivativos $\delta E_p / \delta w$ e depois obtém-se a soma total do algoritmo: $\frac{\partial E}{\partial w} = \sum_p \frac{\partial E_p}{\partial w}$ ⇒ **cálculo correto do gradiente.**

Aprendizado Incremental

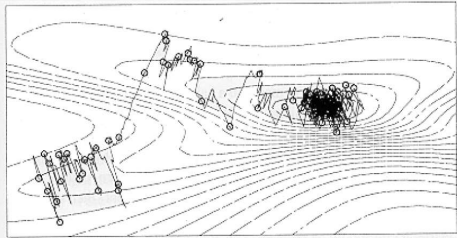
Verdadeiro gradiente x Gradiente de um padrão



- O gradiente de um único padrão pode ser visto como uma **estimativa ruidosa** do verdadeiro gradiente;
- Ele pode ter **projeções negativas** sobre o verdadeiro gradiente;
- Na média, ele se move "**downhill**";
- Quando $\eta \ll 1$, os dois métodos se aproximam.

Aprendizado Incremental

• Aprendizado Incremental



- Na apresentação randômica dos padrões, a descida não é mais suave;
- Na média diminui o erro mas pode, eventualmente, aumentá-lo;
- No final do treinamento η deve ser pequeno para evitar oscilações.

Avaliação do Algoritmo

- Apesar do grande sucesso do Back Propagation, existem alguns problemas:
 - definição do **tamanho da rede**;
 - problema de **paralisia** da Rede Neural;
 - problema de ficar preso em um **mínimo local**.

Batch x Incremental

- O modo **Batch** necessita de menos atualizações de pesos → **Tende a ser mais rápido**
- **Batch** fornece uma medida mais precisa da mudança necessária dos pesos
- **Incremental** tem menos chance de ficar preso em um mínimo local devido à apresentação aleatória dos padrões → **natureza estocástica de busca no espaço de pesos** → **Tende a ser mais rápido se o conjunto de treinamento for grande e ruidoso.**



A eficiência dos dois métodos depende do problema em questão

Tamanho da Rede Neural

- Define o **número de camadas escondidas** e o **número de processadores** em cada uma dessas camadas.
- ↓
- Compromisso entre **Convergência** e **Generalização**.
 - Também conhecido como **“bias and variance dilemma”**.

Avaliação do Algoritmo

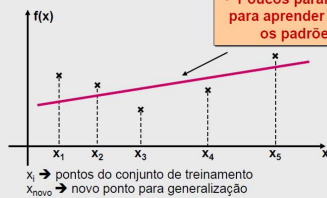
- Apesar do grande sucesso do Back Propagation, existem alguns problemas:
 - definição do **tamanho da rede**;
 - problema de **paralisia** da Rede Neural;
 - problema de ficar preso em um **mínimo local**.

Convergência

- É a capacidade da Rede Neural de **aprender todos os padrões** do conjunto de treinamento.
- ↓
- Se a rede neural for **pequena**, **não** será capaz de **armazenar** todos os padrões necessários.
 - Isto é, a rede não deve ser **rígida** a ponto de não **modelar fielmente os dados**.

Convergência

- É a capacidade da Rede Neural de **aprender todos os padrões** do conjunto de treinamento.



Generalização

• Métricas:

- ② Upadhyaya & Eryurek → O número de **parâmetros** necessários para codificar **P** padrões binários é $\log_2 P$

número de pesos

$$W \leq N_{in} \cdot \log_2 P$$

número de entradas

número de padrões

Generalização

- Se a rede for **muito grande (muitos parâmetros = pesos)**, **não responderá corretamente** aos padrões nunca vistos.
- Isto é, a rede não deve ser **excessivamente flexível** a ponto de **modelar também o ruído**



Generalização

• Métricas:

- ③ Baum-Haussler → $N \geq \frac{w}{\epsilon}$ $0 < \epsilon \leq 1/8$
- número de padrões
- número de sinapses
- Erro desejado no teste (erro_{trein} = $\epsilon/2$)
- (generalização x precisão)

Generalização

• Métricas:

- ① Hecht-Nielsen → Usou Kolmogorov Theorem: qualquer função de **n** variáveis pode ser representada por **2n+1** funções de uma variável

número de PEs na camada escondida

$$N_{hidden} \leq 2N_{in} + 1$$

número de entradas

Generalização

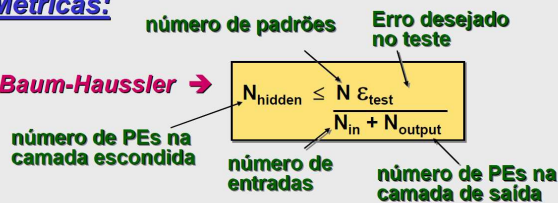
• Métricas:

- ③ Baum-Haussler → $N \geq \frac{w}{\epsilon}$ $0 < \epsilon \leq 1/8$
- número de padrões
- número de sinapses
- Erro desejado no teste (erro_{trein} = $\epsilon/2$)
- (generalização x precisão)
- $N_{hidden} = \frac{w}{N_{in} + N_{out}}$

Generalização

• Métricas:

④ Baum-Haussler →



Generalização

- Adiciona um termo denominado **weight decay**
- Usado para ajustar a complexidade da rede à dificuldade do problema;
- ⇒ Se a estrutura da rede for muito complexa, pode-se remover alguns pesos sem aumentar o erro significativamente;
- ⇒ o método fornece aos pesos uma tendência de se dirigir ao valor zero, reduzindo a sua magnitude um pouco a cada iteração.

Generalização

Resumo sobre número de Processadores Escondidos

- Nunca escolha $h > 2i$;
- Pode-se armazenar p padrões de i elementos em $i \log_2 p$ processadores escondidos
 - * nunca utilize mais do que esse limite
 - * para uma boa generalização utilize um número consideravelmente menor
- Assegure-se que se tem pelo menos $1/\epsilon$ vezes mais padrões do que w
- Número de padrões $> 10 \times$ número de pesos

Weight Decay

- Adiciona um termo denominado **weight decay**

$$\Delta w(t) = -\eta \frac{\partial E}{\partial w}(t) - \rho w(t)$$

onde $0 \leq \rho \ll 1$

- Se $\delta E / \delta w_i = 0 \Rightarrow w_i$ irá diminuir até zero;
- caso contrário, os dois termos irão se equilibrar em um certo ponto, evitando que $w_i \rightarrow 0$

Generalização

Técnicas para Reduzir a complexidade da rede:

- **Weight Decay**
- **Análise de Sensibilidade**

Weight Decay

- Adiciona um termo denominado **weight decay**

$$\Delta w(t) = -\eta \frac{\partial E}{\partial w}(t) - \rho w(t)$$

onde $0 \leq \rho \ll 1$

- Equivale a modificar a definição de **E** incluindo um **termo de penalidade** correspondente à magnitude dos pesos da rede.

$$E = \frac{1}{2} \sum_{i=1}^k (d_i - y_i)^2 + \frac{1}{2} \lambda \|w\|^2$$

Generalização

- Análise de Sensibilidade da Saída:

- **Retira-se** uma variável de entrada ou elementos processadores;
- **Treina-se** novamente a rede;
- **Verifica-se a variação do erro** de saída.



- Se a retirada do elemento não causar grande variação $\Rightarrow$ a rede é pouco sensível a este elemento, podendo ser retirado sem perda da capacidade de modelar os dados.

Avaliação do Algoritmo

- Apesar do grande sucesso do Back Propagation, existem alguns problemas:

- definição do **tamanho da rede**;
- problema de **paralisia** da Rede Neural;
- problema de ficar preso em um **mínimo local**.

Avaliação do Algoritmo

- Apesar do grande sucesso do Back Propagation, existem alguns problemas:

- definição do **tamanho da rede**;
- problema de **paralisia** da Rede Neural;
- problema de ficar preso em um **mínimo local**.

Avaliação do Algoritmo

- **Como evitar paralisia da Rede?**

- Deve-se escolher valores de **pesos** e **bias** uniformemente distribuídos dentro de um **intervalo pequeno**;
- Os **neurônios** devem, inicialmente, operar na sua **região linear**;
- O **número de processadores** na(s) camada(s) escondida(s) deve ser **pequeno**.

Problema de Paralisia

Com o treinamento, os **pesos** podem alcançar valores muito **grandes** ($w_{ji} \nearrow$)

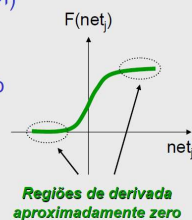


A **soma ponderada** de cada processador torna-se também muito **grande** ($net_j = \sum x_i \cdot w_{ji} + \theta_j \nearrow$)

$$F'(net_j) \sim 0$$



$\Delta w_{ji} \sim 0 \Rightarrow$ A Rede Neural não consegue aprender



Avaliação do Algoritmo

- Apesar do grande sucesso do Back Propagation, existem alguns problemas:

- definição do **tamanho da rede**;
- problema de **paralisia** da Rede Neural;
- problema de ficar preso em um **mínimo local**.

Problema do Mínimo Local

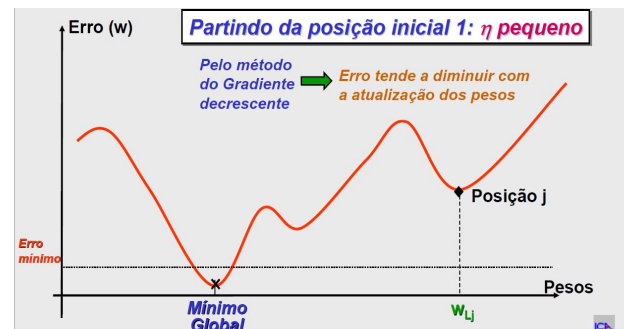
De acordo com o método do gradiente decrescente, tem-se: $\Delta w_{ji} = \eta \cdot s_i \cdot e_j$



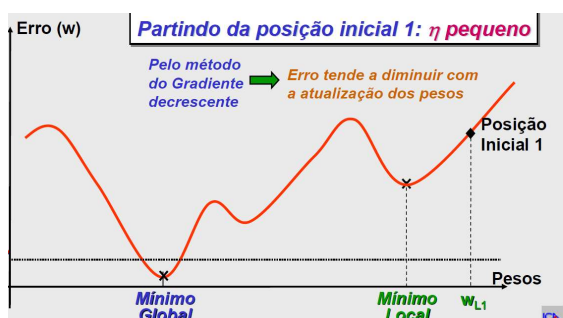
Taxa de aprendizado
 $0.05 \leq \eta \leq 0.75$

A taxa de aprendizado não deve ser **nem muito pequena** → **treinamento lento**,
nem muito grande → **oscilações**

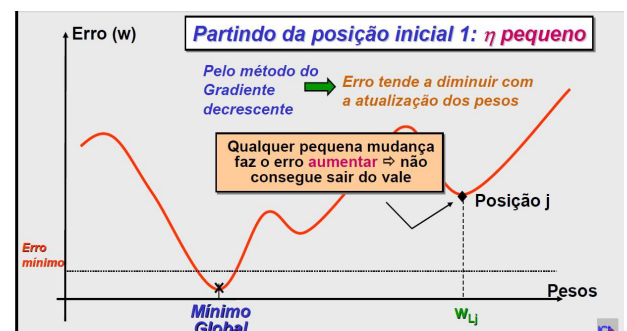
Problema do Mínimo Local



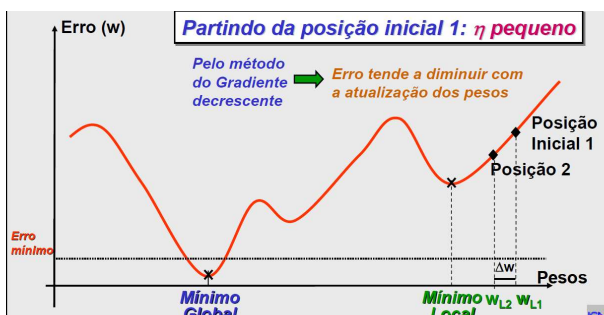
Problema do Mínimo Local



Problema do Mínimo Local



Problema do Mínimo Local



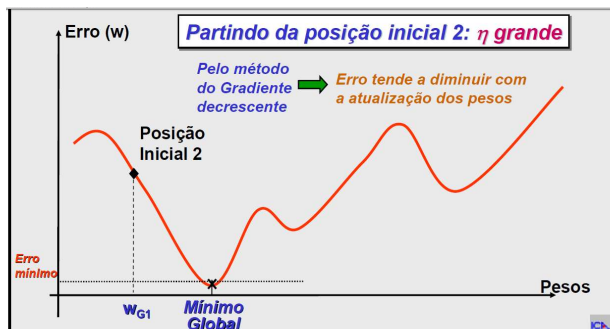
Problema do Mínimo Local

Com η pequeno e dependendo da inicialização dos pesos (feita de forma aleatória) não é possível calcular um Δw que faça a Rede Neural sair do **Minimo Local**



A Rede Neural **não consegue aprender com a precisão especificada (Erro Mínimo)!**

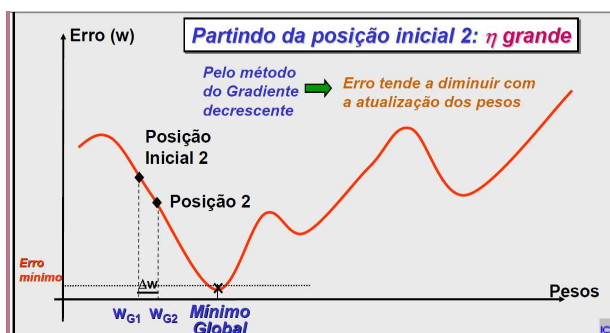
Problema do Mínimo Local



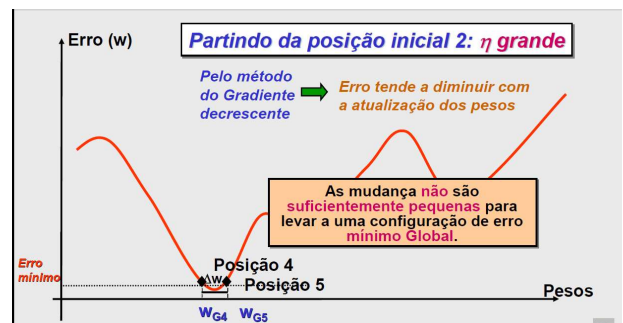
Problema do Mínimo Local



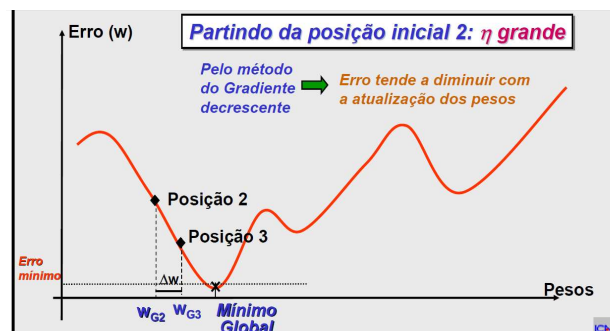
Problema do Mínimo Local



Problema do Mínimo Local



Problema do Mínimo Local



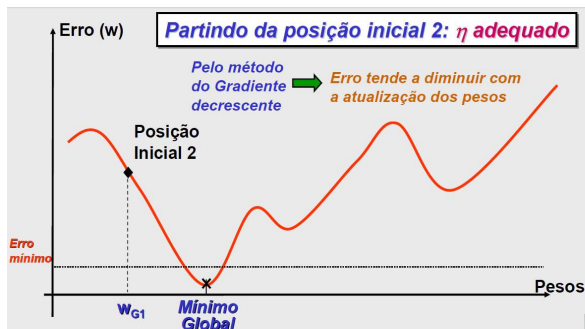
Problema do Mínimo Local

Quando η é **grande**, a Rede Neural pode nunca conseguir chegar ao Mínimo Global pois os valores de Δw são **grandes**. → **Oscilações**.

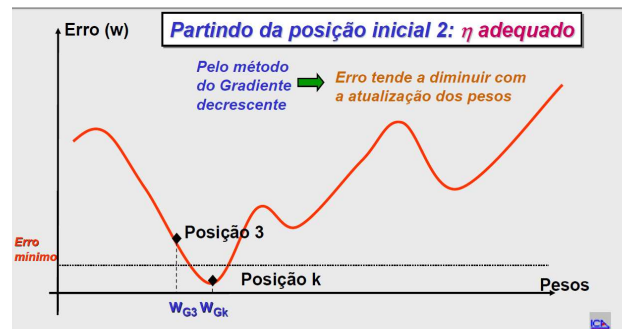


A Rede Neural também não consegue aprender com a precisão especificada (fica oscilando em torno do mínimo global)!

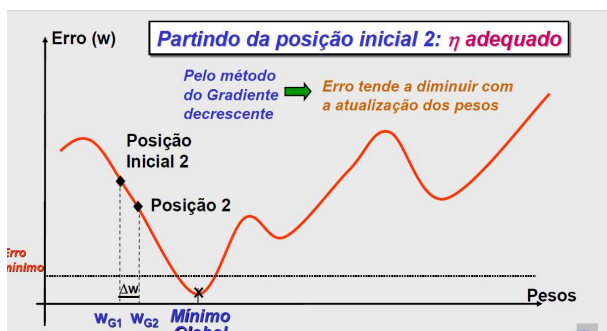
Problema do Mínimo Local



Problema do Mínimo Local



Problema do Mínimo Local

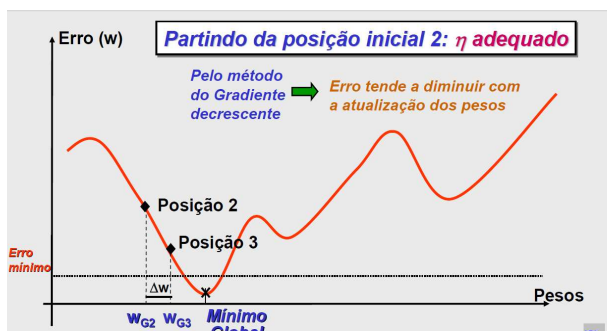


Problema do Mínimo Local

• Soluções:

- Utilizar taxa de aprendizado (η) adaptativa;
- Utilizar o termo de Momento (α).

Problema do Mínimo Local



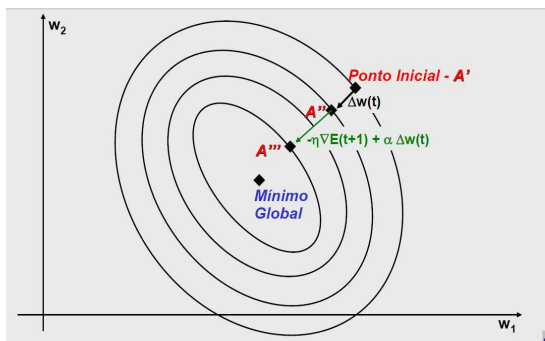
Termo de Momento (α)

Uma forma de **diminuir** a possibilidade de **oscilações** com valores de η maiores é incluir o **Termo de Momento** na atualização dos pesos sinápticos

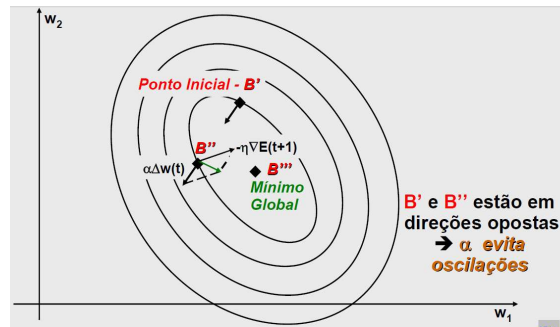
$$\Delta w_{ji}(t+1) = \eta \cdot s_i \cdot e_j + \alpha \cdot \Delta w_{ji}(t)$$

Termo proporcional ao valor anterior de atualização do peso
 $0 < \alpha < 0.9$

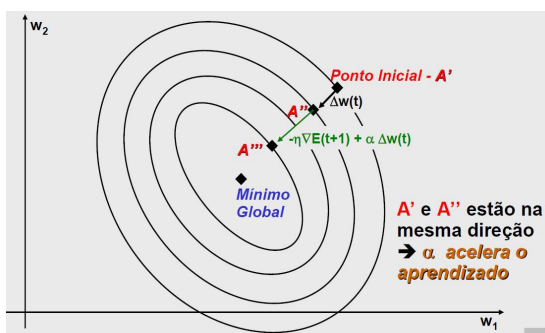
Termo de Momento (α)



Termo de Momento (α)

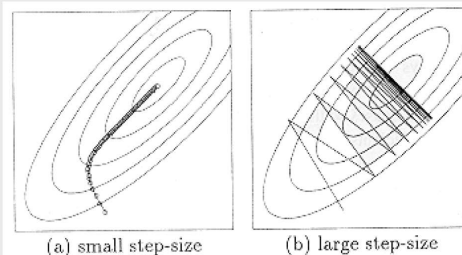


Termo de Momento (α)

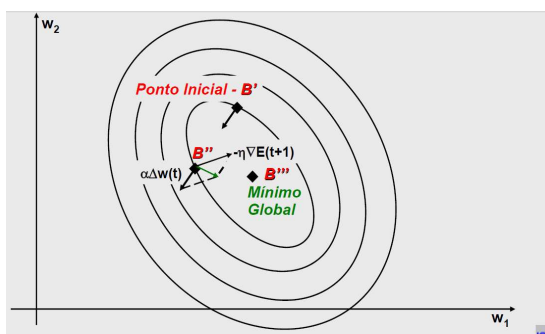


Termo de Momento (α)

Aprendizado em Batch sem Termo de Momento

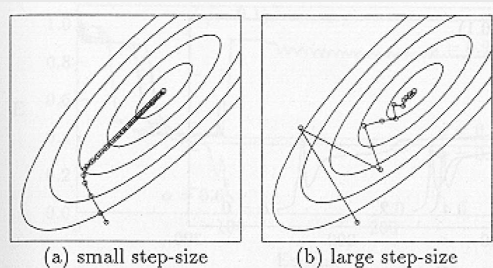


Termo de Momento (α)



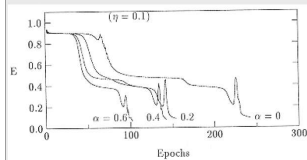
Termo de Momento (α)

Aprendizado em Batch com Termo de Momento

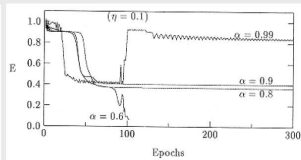


Termo de Momento (α)

Aprendizado em **Batch** com **Termo de Momento**
Simulações com a mesma configuração inicial de pesos



Conforme α aumenta
 $\Rightarrow$ converge mais rápido
(para valores pequenos)



Conforme α aumenta mais
 $\Rightarrow$ pode oscilar

Fim do Back Propagation